Documentación Técnica del Sistema

Proyecto: *Vida Natural* — Plataforma Web de E-Commerce y Gestión Nutricional

Ficha Técnica del Proyecto

- **Asignatura Académica:** Algoritmos y Estructuras de Datos III (ALED III)
- **Carrera:** Analista de Sistemas
- **Autor / Equipo de Desarrollo:** Equipo de Proyecto Vida Natural
- **Tecnología Principal:** Angular 17 (v17.3.0)
- **Lenguaje:** TypeScript 5.4.2
- **Gestor de Paquetes y Runtime:** Node.js v20.x / npm v10.x
- **Arquitectura de Compilación:** `@angular-devkit/build-angular:application` (Vite / esbuild)
- **Librería de Componentes y UX:** Angular Material v17.3.10 & Angular CDK v17.3.10
- **Estilos y Maquetación:** CSS3 Modular con Variables Semánticas Custom + Bootstrap 5.3.2 (Grid & Utilities) + Bootstrap Icons 1.11.3
- **Estado de Compilación:** SPA (Single Page Application) desplegable en Netlify

---

1. Introducción y Objetivos del Sistema

El sistema **Vida Natural** es una aplicación web de una sola página (Single Page Application - SPA) orientada a la comercialización minorista de productos naturales, orgánicos y suplementos dietarios, complementada con un motor de consulta nutricional conectado a bases de datos internacionales.

1.1 Objetivos Generales

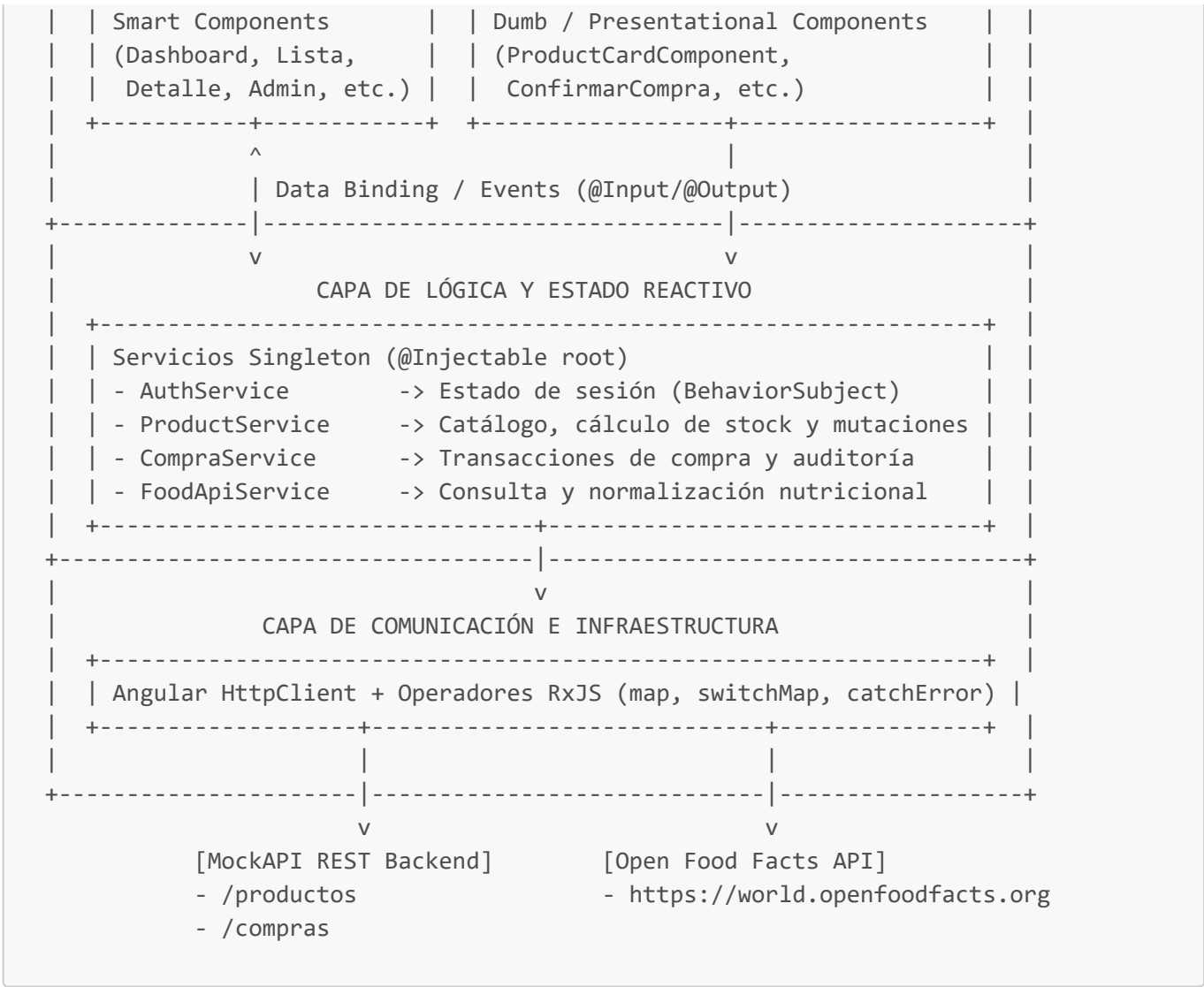
1. Implementar una solución de software escalable, modular y mantenible basada en el ecosistema **Angular 17**, respetando los estándares de ingeniería de software para aplicaciones empresariales.
2. Garantizar la separación de responsabilidades (*Separation of Concerns*) entre la capa de presentación, la lógica de negocio y el consumo de servicios web desacoplados.
3. Proporcionar un control de acceso robusto mediante **Control de Acceso Basado en Roles (RBAC)** aplicado a nivel de enrutamiento del lado del cliente.
4. Optimizar el rendimiento y el tiempo de carga inicial (*First Contentful Paint* - FCP) mediante la técnica de carga diferida (*Lazy Loading*).

---

2. Arquitectura de Software y Patrones de Diseño

La solución adopta una **Arquitectura en Capas Orientada a Componentes y Flujos Reactivos**, regida por los principios de diseño SOLID y los lineamientos de buenas prácticas de la documentación oficial de Angular.





2.1 Patrones de Diseño Implementados

1. **Singleton Service Pattern:** Todos los servicios de infraestructura y negocio (`AuthService`, `ProductService`, `CompraService`, `FoodApiService`, `AuthGuard`, `RoleGuard`) declaran `@Injectable({ providedIn: 'root' })`. Esto garantiza una única instancia en memoria para toda la aplicación, permitiendo un estado consistente y evitando duplicaciones de caché o conexiones.

2. **Observer / Reactive Streams Pattern (RxJS):** La comunicación de eventos asíncronos y la gestión de estado se apoya en `Observable`, `Subject` y `BehaviorSubject`.

    - `AuthService.usuarioActual`: Un `BehaviorSubject<User | null>` almacena el usuario autenticado, notificando automáticamente a suscriptores activos (ej. `NavbarComponent`) ante mutaciones del ciclo de sesión (login/logout).
    - `ProductosListaComponent.busquedaSubject`: Emplea una canalización reactiva compuesta por `debounceTime(300)` y `distinctUntilChanged()` para regular la carga de filtrado y evitar sobreprocesamiento por pulsación de teclado.

3. **Container / Presentational Component Pattern (Smart & Dumb):**

    - **Presentacionales (Dumb):** `ProductCardComponent` carece de inyecciones de servicios o conocimiento del estado global. Recibe su modelo vía `@Input() producto!: Product` y

delega acciones al contenedor mediante `@Output() verMas = new EventEmitter<Product>()`.
    - **Contenedores (Smart):** `DashboardComponent`, `ProductosListaComponent` y `ProductoDetalleComponent` gestionan suscripciones a servicios, navegación imperativa (`Router`), cálculo de métricas y orquestación de diálogos modales.

4. **Gatekeeper / Guard Pattern:** Implementación de la interfaz `CanActivate` de Angular Router para interceptar la navegación y aplicar reglas de seguridad perimetrales antes de que los componentes o módulos sean instanciados.

5. **Adapter / Data Transfer Object (DTO) Transformation:** En `FoodApiService`, los objetos heterogéneos provenientes de la API pública de *Open Food Facts* (`ProductoCrudo`) son saneados y transformados mediante operadores funcionales (`pipe(map(...))`) en la interfaz unificada `AlimentoExterno`. Esto aísla a los componentes visuales de inconsistencias o cambios en el esquema del proveedor externo.

---

3. Estructura Detallada del Repositorio

El proyecto se organiza bajo una estructura modular orientada a dominios y responsabilidades claras:

```
src/
├── _redirects                    # Regla de reescritura para enrutamiento SPA en Netlify
├── favicon.ico                   # Favicon del sistema
├── index.html                    # Host document con tipografía Poppins y Bootstrap
├── main.ts                       # Punto de entrada de bootstrapping (bootstrapModule)
├── styles.css                    # Sistema de diseño global y tokens semánticos CSS
├── environments/                 # Variables de entorno por ambiente
│   ├── environment.prod.ts       # Endpoint de producción (MockAPI)
│   └── environment.ts            # Endpoint de desarrollo (MockAPI)
└── app/
    ├── app-routing.module.ts     # Enrutador principal y configuración de Lazy Loading
    ├── app.component.html        # Shell de la aplicación (<app-navbar> + <router-outlet>)
    ├── app.component.ts          # Componente raíz
    ├── app.module.ts             # Módulo raíz (Eager Loading, providers globales)
    ├── guards/                   # Centinelas de protección de rutas
    │   ├── auth.guard.ts         # Verificación de sesión activa en localStorage
    │   └── role.guard.ts         # Verificación de autorización RBAC (admin vs usuario)
    ├── models/                   # Definiciones de tipos e interfaces TypeScript
    │   ├── compra.model.ts       # Entidad Compra registrada en MockAPI
    │   ├── product.model.ts      # Entidad Producto del catálogo
    │   └── user.model.ts         # Entidad Usuario y credenciales de autenticación
```

```
    ├── services/                  # Capa de lógica de negocio y comunicación HTTP
    │   ├── auth.service.ts        # Gestión de autenticación y estado reactivo de
sesión
    │   ├── compra.service.ts      # Cliente HTTP para endpoint /compras
    │   ├── food-api.service.ts    # Cliente HTTP para Open Food Facts
    │   └── product.service.ts     # Cliente HTTP y fallback local para /productos
    ├── shared/                    # Módulo compartido de componentes
reutilizables
    │   ├── shared.module.ts       # Módulo exportador de elementos comunes
    │   ├── navbar/                # Barra de navegación principal y control de
sesión
    │   └── product-card/          # Tarjeta presentacional desacoplada
    └── pages/                     # Vistas y flujos de navegación del sistema
        ├── login/                 # Formulario reactivo de autenticación
        ├── dashboard/             # Panel métrico y destacados
        ├── buscar-alimentos/      # Buscador externo de información nutricional
        ├── mis-compras/           # Historial transaccional del cliente
autenticado
        ├── admin-compras/         # Auditoría de compras y métricas globales de
venta
        └── productos/             # Módulo cargado con LAZY LOADING
            ├── productos.module.ts          # Feature Module con importaciones
diferidas
            ├── productos-routing.module.ts  # Rutas internas y jerarquía de
parámetros
            ├── productos-lista/             # Grilla con filtrado multicriterio
reactivo
            ├── producto-detalle/            # Ficha técnica, compra y reposición
de stock
            ├── producto-nuevo/              # Alta de producto (exclusivo admin)
            ├── producto-editar/             # Modificación de producto (exclusivo
admin)
            ├── confirmar-compra/            # Diálogo modal de resumen
transaccional
            └── confirmar-eliminar/          # Diálogo modal de borrado seguro
```

---

4. Sistema de Enrutamiento y Lazy Loading

El enrutamiento está centralizado en `AppRoutingModule` y delegaciones específicas en submódulos mediante `RouterModule.forRoot()` y `RouterModule.forChild()`.

4.1 Matriz de Rutas del Sistema

Ruta	Componente / Módulo	Tipo de Carga	Guards Aplicados	Roles Permitidos
`/`	Redirección a `/login`	Inmediata	Ninguno	Todos (Público)
`/login`	`LoginComponent`	Inmediata	Ninguno	Todos (Público)

Ruta	Componente / Módulo	Tipo de Carga	Guards Aplicados	Roles Permitidos
`/inicio`	`DashboardComponent`	Inmediata	`AuthGuard`	`admin`, `usuario`
`/buscar-alimentos`	`BuscarAlimentosComponent`	Inmediata	`AuthGuard`	`admin`, `usuario`
`/productos`	`ProductosModule`	**Lazy Loading**	`AuthGuard`	`admin`, `usuario`
`/productos/nuevo`	`ProductoNuevoComponent`	**Lazy Loading**	`RoleGuard`	`admin`
`/productos/:id/editar`	`ProductoEditarComponent`	**Lazy Loading**	`RoleGuard`	`admin`
`/productos/:id`	`ProductoDetalleComponent`	**Lazy Loading**	Ninguno (hereda Auth)	`admin`, `usuario`
`/mis-compras`	`MisComprasComponent`	Inmediata	`AuthGuard`	`admin`, `usuario`
`/admin/compras`	`AdminComprasComponent`	Inmediata	`AuthGuard`, `RoleGuard`	`admin`
`**`	Redirección a `/login`	Inmediata	Ninguno	Todos (Comodín)

4.2 Mecánica y Justificación del Lazy Loading

El catálogo de productos representa el subsistema más complejo de la plataforma, incorporando formularios reactivos de edición, validadores avanzados, modales de confirmación (`MatDialog`) y utilidades de retroalimentación (`MatSnackBar`).

```
// Fragmento de app-routing.module.ts
{
  path: 'productos',
  canActivate: [AuthGuard],
  loadChildren: () =>
    import('./pages/productos/productos.module')
      .then(m => m.ProductosModule)
}
```

Ventajas Técnicas Comprobadas:

1. **Reducción de Bundle Inicial:** Durante la carga de `/login` o `/inicio`, el navegador no descarga ni procesa los artefactos de `ProductosModule`, acelerando la interactividad del usuario inicial.
2. **Chunking Autónomo:** El compilador de Angular segrega automáticamente el módulo en un archivo JavaScript separado (ej. `src_app_pages_productos_productos_module_ts.js`), descargado vía

HTTP únicamente cuando se produce la primera navegación hacia `/productos`.

4.3 Precedencia y Resolución de Conflictos de Rutas

En `ProductosRoutingModule`, el orden de declaración de rutas es crítico debido al algoritmo de concordancia secuencial (*First-Match Strategy*) del Router de Angular:

```
const routes: Routes = [
  { path: '', component: ProductosListaComponent },
  { path: 'nuevo', component: ProductoNuevoComponent, canActivate: [RoleGuard],
data: { roles: ['admin'] } },
  { path: ':id/editar', component: ProductoEditarComponent, canActivate:
[RoleGuard], data: { roles: ['admin'] } },
  { path: ':id', component: ProductoDetalleComponent }
];
```

> **Fundamento Técnico:** Si `:id` se definiera antes que `nuevo`, la cadena literal `"nuevo"` sería capturada erróneamente por el enrutador como un parámetro de identificación dinámica (`id = 'nuevo'`), provocando una falla en `ProductoDetalleComponent` al intentar convertir dicho texto a un entero (`NaN`). Por ello, las rutas estáticas preceden estrictamente a las dinámicas.

---

5. Seguridad y Control de Acceso (Guards & RBAC)

La plataforma implementa un esquema de **Role-Based Access Control (RBAC)** validado a través de dos guardias de activación (`CanActivate`).

5.1 Diagrama de Secuencia de Navegación y Evaluación de Guards

```
[Usuario / Navegador]
       |
       |  1. Intenta navegar a: /admin/compras
       v
[Angular Router]
       |
       |  2. Ejecuta canActivate() de AuthGuard
       v
   [AuthGuard]
       |
       |-- ¿authService.isAuthenticated()?
       |   |-- NO: Redirige a /login -> return false
       |   `-- SÍ: return true
       v
[Angular Router]
       |
       |  3. Ejecuta canActivate(route) de RoleGuard
       v
   [RoleGuard]
       |
```

```
        |-- Extrae rolesPermitidos = route.data['roles'] (['admin'])
        |-- Obtiene rolActual = authService.getRole()
        |-- ¿rolesPermitidos.includes(rolActual)?
        |   |-- NO: Redirige a /inicio -> return false
        |   `-- SÍ: return true
        v
[Angular Router]
        |
        |  4. Carga y renderiza AdminComprasComponent
        v
[Usuario / Navegador]
```

5.2 Implementación Rigurosa de los Guards

A. `AuthGuard` **(**`src/app/guards/auth.guard.ts`**)**

```typescript
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.authService.isAuthenticated()) {
      return true;
    }
    this.router.navigate(['/login']);
    return false;
  }
}
```

B. `RoleGuard` **(**`src/app/guards/role.guard.ts`**)**

```typescript
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  constructor(private authService: AuthService, private router: Router) {}

  canActivate(route: ActivatedRouteSnapshot): boolean {
    const rolesPermitidos: string[] = route.data['roles'];
    const rolActual = this.authService.getRole();

    if (rolActual && rolesPermitidos.includes(rolActual)) {
      return true;
    }

    this.router.navigate(['/inicio']);
    return false;
  }
}
```

5.3 Consideraciones de Seguridad en el Frontend

- **Persistencia de Sesión:** Se gestiona a través de la clave serializada `'vn_user'` en el almacenamiento local del navegador (`localStorage`), restaurándose de forma síncrona en el constructor del `AuthService` para garantizar permanencia ante refrescos de página (`F5`).
- **Principio de Mínimo Privilegio en UI:** Las directivas condicionales `*ngIf="esAdmin"` ocultan o exponen botones operativos (alta, edición, borrado, reposición de stock y enlaces de navegación administrativa). Si un usuario manipula el DOM o escribe la URL directamente en el navegador, el `RoleGuard` aborta la navegación y lo redirige de inmediato a `/inicio`.
- **Aviso Académico de Seguridad:** En un entorno productivo de misión crítica, la seguridad en el cliente (SPA) es complementaria. Toda operación de escritura (POST, PUT, DELETE) debe ser indefectiblemente validada y autorizada por el backend mediante verificación de firmas criptográficas (Tokens JWT / OAuth2).

---

6. Lógica de Negocio, Estado y Servicios de Integración

6.1 `AuthService` (`src/app/services/auth.service.ts`)

- Mantiene el estado canónico del usuario activo mediante `BehaviorSubject<User | null>`.
- Proporciona flujos reactivos (`usuario$`) para elementos dinámicos de la interfaz y accesores síncronos (`usuarioActualValor`, `getRole()`, `esAdmin()`).
- Simula latencia de red controlada (600 ms) en el método `login()` para replicar la experiencia de un endpoint HTTP real y permitir la visualización de animaciones de progreso (`MatProgressSpinner`).
- Administra el cierre de sesión atómico (`logout()`): limpia `localStorage`, emite `null` en el observable y navega a `/login`.

6.2 `ProductService` (`src/app/services/product.service.ts`)

Provee la abstracción completa del catálogo comercial con soporte dual: conexión a MockAPI REST (`environment.apiUrl`) y mecanismo de respaldo local (*in-memory fallback*).

Transacción Atómica de Compra (`comprarProducto`):

Garantiza la consistencia del inventario mediante encadenamiento de observables con `switchMap`:

1. Ejecuta una petición `getProductById(id)` para validar el estado más reciente del recurso en el servidor.
2. Comprueba disponibilidad y valida que la cantidad solicitada no sobrepase el stock actual (`cantidad <= stock`).
3. Calcula la diferencia (`nuevoStock = stock - cantidad`) y actualiza la bandera `disponible = nuevoStock > 0`.
4. Envía una mutación `PUT` a la API con los atributos modificados.

Reposición de Inventario (`agregarStock`):

Permite a los administradores reabastecer unidades agotadas. Automáticamente reactiva la disponibilidad comercial (`disponible = true`) al recibir un stock positivo.

6.3 `CompraService` (`src/app/services/compra.service.ts`)

Encargado de la persistencia de las transacciones generadas en la tienda mediante el endpoint `/compras`.

- **Integridad de Datos:** Al registrar la compra, los datos de identidad (`usuarioEmail` y `usuarioNombre`) son inyectados programáticamente desde el `AuthService`, impidiendo que el cliente adultere su autoría.
- **Segmentación por Usuario:** El método `getComprasUsuario(email)` consulta `/compras?usuarioEmail=xxx`, delegando el filtrado al backend mock.
- **Auditoría Global:** `getTodasLasCompras()` recupera el universo completo de compras para el módulo de administración, permitiendo consolidar estadísticas de venta global.

6.4 `FoodApiService` (`src/app/services/food-api.service.ts`)

Servicio de integración con la API REST pública de *Open Food Facts* (`https://world.openfoodfacts.org/cgi/search.pl`).

- **Optimización de Ancho de Banda:** Restringe la respuesta del servidor mediante el parámetro `fields=product_name,brands,categories,image_url,nutriscore_grade,nutriments`, evitando la descarga de megabytes de metadatos no requeridos.
- **Tolerancia a Fallos:** Intercepta códigos de error HTTP mediante `catchError` y emite mensajes amigables y tipados en español según la naturaleza del incidente (código 0 para fallas de conectividad local, >= 500 para indisponibilidad del servidor externo).

---

7. Interfaz de Usuario, Experiencia (UX) e Internacionalización

7.1 Sistema de Diseño Propio "Vida Natural"

El archivo `src/styles.css` define una paleta cromática orgánica basada en variables semánticas:

- `--vn-verde: #2d7a4f`: Color de marca principal, evocador de salud y naturaleza.
- `--vn-verde-hover: #1b5e35`: Estados interactivos de foco y hover.
- `--vn-verde-suave: #e8f5e9`: Fondos de badges y contenedores de éxito.
- `--vn-beige: #f9f5f0`: Fondo general de la plataforma, mitigando la fatiga visual.
- `--vn-borde: #d4e6d8`: Delimitadores suaves para cards y tablas.

7.2 Componentes de Angular Material y Accesibilidad

- **Modales de Confirmación (`MatDialog`):** Implementados en `ConfirmarCompraComponent` y `ConfirmarEliminarComponent`. Previenen errores accidentales de compra o eliminación irreversible de datos mediante cuadros de diálogo bloqueantes con paso de datos tipados (`MAT_DIALOG_DATA`).
- **Retroalimentación Toast (`MatSnackBar`):** Notificaciones flotantes no invasivas que confirman operaciones exitosas (ej. "¡Compra realizada correctamente!") o advierten errores de comunicación con duración temporal calibrada (3000 a 3500 ms).
- **Indicadores de Carga (`MatProgressSpinner`):** Spinners integrados en botones y vistas de carga para proporcionar retroalimentación visual durante peticiones asíncronas.

7.3 Internacionalización y Localización (i18n / l10n)

En `AppModule` se registra el locale oficial de la República Argentina:

```
import { registerLocaleData } from '@angular/common';
import localeEsAr from '@angular/common/locales/es-AR';
registerLocaleData(localeEsAr);

providers: [
  { provide: LOCALE_ID, useValue: 'es-AR' }
]
```

Esto habilita el comportamiento nativo de los pipes de Angular en todas las plantillas:

- `CurrencyPipe`: `{{ compra.total | currency:'ARS':'symbol':'1.0-0':'es-AR' }}` formatea los montos monetarios según el estándar argentino (`$ 1.850`).
- `DatePipe`: `{{ compra.fecha | date:'dd/MM/yyyy HH:mm' }}` estandariza las marcas temporales en formato día/mes/año con hora y minutos.

---

8. Estrategia de Compilación y Despliegue en Producción

8.1 Motor de Compilación de Angular 17

El archivo `angular.json` utiliza el nuevo constructor `@angular-devkit/build-angular:application`, que combina **Vite** para el servidor de desarrollo y **esbuild** para empaquetado de producción, reduciendo los tiempos de transpilación y optimizando la eliminación de código muerto (*Tree-Shaking*).

8.2 Despliegue en Netlify y Resolución del Enrutamiento SPA

Las aplicaciones de una sola página gestionan las rutas dinámicamente en el cliente mediante la API History de HTML5. Sin embargo, al refrescar la página o acceder directamente a una URL como `https://sitio.netlify.app/productos`, el servidor web estático buscaría un archivo físico `/productos/index.html`, devolviendo un error HTTP 404.

Para solucionar este comportamiento, el proyecto incluye dos configuraciones complementarias:

1. **`src/_redirects`:**

   ```
   /*    /index.html   200
   ```

   Indica al servidor de Netlify que cualquier solicitud entrante (`/*`) debe reescribirse internamente hacia `/index.html` con código de estado HTTP 200, delegando el control del enrutamiento al motor de Angular.

2. **`netlify.toml`:**

   ```
   [build]
     command = "npm run build"
   ```

```
    publish = "dist/vida-natural/browser"

  [build.environment]
    NODE_VERSION = "20"
    CI = "true"
```

Configura el entorno de ejecución en Node 20 y establece la carpeta `dist/vida-natural/browser` como la raíz pública de distribución generada por el builder de Angular 17.

---

9. Conclusiones de Ingeniería y Calidad del Software

La arquitectura implementada en **Vida Natural** satisface holgadamente las exigencias académicas y las normativas de la industria:

1. **Mantenibilidad:** El desacoplamiento estricto de componentes y la delegación de responsabilidades a servicios especializados asegura que futuros requerimientos (ej. pasarelas de pago reales o nuevos proveedores nutricionales) se integren sin alterar la base de código existente.
2. **Robustez y Seguridad:** El control de rutas mediante `AuthGuard` y `RoleGuard`, junto con validaciones síncronas en formularios reactivos y chequeos atómicos de inventario, minimiza la probabilidad de inconsistencias de datos o accesos no autorizados.
3. **Experiencia de Usuario (UX):** La combinación de carga diferida (Lazy Loading), búsqueda reactiva con cancelación de ruido (*debounce*), internacionalización de formatos y retroalimentación contextual mediante Angular Material proporciona una navegación fluida, profesional y accesible.